lr_2000

November 8, 2026

[9]:
```
import pandas as pd
```

[10]:
```
df = pd.read_csv("./data/train_starter_2000.csv")
```

[11]:
```
df.head()
```

[11]:

	id	text	label
0	1	Cipriani S.A. is an Italian hotel and leisure …	0
1	3	"Petit Beurre -- Butter Cookies\n\n1 cup (2 st…	1
2	23	Serra was born in San Francisco, California to…	1
3	24	Medical image segmentation is inherently an am…	0
4	29	Introduction\n\nCASSOD-Net (Centralized Attent…	1

[12]:
```
from sklearn.model_selection import train_test_split

X = df["text"]
y = df["label"]

X,y
```

[12]:
```
(0       Cipriani S.A. is an Italian hotel and leisure …
 1       "Petit Beurre -- Butter Cookies\n\n1 cup (2 st…
 2       Serra was born in San Francisco, California to…
 3       Medical image segmentation is inherently an am…
 4       Introduction\n\nCASSOD-Net (Centralized Attent…
                              …
 1995    We propose variational reaction-diffusion syst…
 1996    When I see a bus on the road, it fills me with…
 1997    So as is customary for all my playthroughs for…
 1998    Lusaka, ZAMBIA (AP) - The government in this s…
 1999    3/4 cup whole almonds (see Notes)\nVegetable-o…
 Name: text, Length: 2000, dtype: str,
 0       0
 1       1
 2       1
 3       0
 4       1
        ..
```

```
1995    1
1996    1
1997    0
1998    1
1999    0
Name: label, Length: 2000, dtype: int64)
```

[13]:
```
# Split data into train and test sets

RANDOM_SEED = 42
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2,␣
 ↪random_state=RANDOM_SEED, stratify=y)

X_train, X_test, y_train, y_test
```

[13]:
```
(937     I don't think it's appropriate or ethical to i…
 959     Marcel Hug (born August 10, 1968) is a Swiss a…
 1015    The main aim of this paper is twofold. First w…
 1216    John Samuel Rowell (April 1, 1825 - October 21…
 1010    This film was nominated in 11 categories at th…
                              …
 470     Ingredients:\n\n- 1/2 cup butter\n- 1 cup whit…
 921     So about 20 minutes after doing a delivery and…
 487     Asset-based lending is a type of lending that …
 595     In this paper, we present a novel approach to …
 1041    Pierson Fodé is an American actor and model. H…
Name: text, Length: 1600, dtype: str,
 1501    6 tablespoons unsalted butter (3/4 stick)\n1 c…
 43      1 1/4 cups all purpose flour\n1 pinch salt\n1/…
 173     Ingredients:\n1 cup (2 sticks) butter, softene…
 952     Xóchitl Castañeda is the Founding Director of …
 995     1 squash, weighing about 2lbs, chopped into ch…
                              …
 1267    Hey everyone,\n\nI'm a 20 year old guy who is …
 1639    Alexei Leonidovich Kudrin (; born 12 October 1…
 201     A controversial film starring Hollywood actor …
 1309    This report provides an overview of the curren…
 991     https://arxiv.org/abs/2010.10720\n\n> "Robust …
Name: text, Length: 400, dtype: str,
937     1
959     1
1015    0
1216    0
1010    1
       ..
470     1
921     0
```

```
487     1
595     1
1041    1
Name: label, Length: 1600, dtype: int64,
1501    0
43      0
173     1
952     0
995     0
       ..
1267    1
1639    0
201     0
1309    0
991     1
Name: label, Length: 400, dtype: int64)
```

[14]:
```
X_train.shape, y_train.shape
```

[14]:
```
((1600,), (1600,))
```

[15]:
```
# try two approaches: tf-idf + logistic regression vs embedding + logistic␣
 ↪regression

# 1) TF-IDF + logistic regression

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    roc_auc_score,
)


pipeline = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            ngram_range=(1, 2),
            max_features=50000,
            sublinear_tf=True,
        ),
    ),
```

```
    (
        "classifier",
        LogisticRegression(
            random_state=RANDOM_SEED,
            C=1.0,
            max_iter=1000,
        ),
    ),
])

pipeline.fit(X_train, y_train)

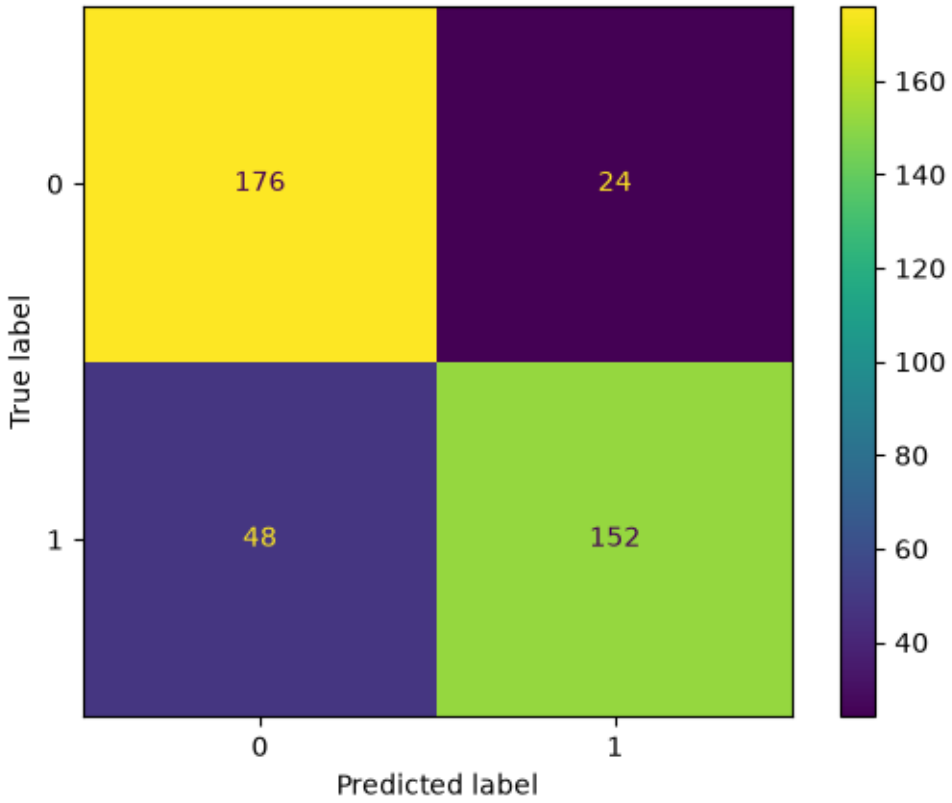
y_pred = pipeline.predict(X_test)
y_probability = pipeline.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_probability)

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8200
F1 score: 0.8085
ROC-AUC: 0.8922
```



[16]:
```
import numpy as np
import ollama

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    roc_auc_score,
)


class OllamaEmbeddingTransformer(BaseEstimator, TransformerMixin):
    """Embed texts with a model served by the local Ollama daemon.

    Ollama answers a single /api/embed call with an error once the combined
    input gets too large (measured here: ~310 KB of text, i.e. roughly
    70k tokens, fails; 200 KB is fine). One batch of 32 short documents
```

```
    is ~40 KB, so there is plenty of headroom.
    Sending all 1600 training texts in one request therefore fails with
    "ResponseError: Post http://127.0.0.1:<port>/v1/tokenize ... connection
    reset by peer", so we embed in small batches instead.
    """

    def __init__(self, model="embeddinggemma-2:270m", batch_size=32):
        self.model = model
        self.batch_size = batch_size

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        texts = np.asarray(X, dtype=str).tolist()

        vectors = []
        for start in range(0, len(texts), self.batch_size):
            batch = texts[start : start + self.batch_size]
            response = ollama.embed(model=self.model, input=batch)
            vectors.extend(response["embeddings"])

        return np.asarray(vectors, dtype=np.float32)


pipeline = Pipeline([
    ("embedding", OllamaEmbeddingTransformer()),
    (
        "classifier",
        LogisticRegression(
            random_state=RANDOM_SEED,
            C=1.0,
            max_iter=1000,
        ),
    ),
])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
y_probability = pipeline.predict_proba(X_test)[:, 1]

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()

accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```python
roc_auc = roc_auc_score(y_test, y_probability)

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 score: {f1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
```

```
Accuracy: 0.8100
F1 score: 0.8041
ROC-AUC: 0.8900
```

